

Anexo

Acceso y uso del producto

**Diseño y validación de una plataforma segura de APIs para
integración bancaria**

Autor	Antonio Martos Gavilán
Titulación	Grado de Ingeniería Informática
Área	Sistemas de Gestión del Conocimiento
Tutor	Oscar Martín Salas
Fecha	08/06/2026
Versión	1.2 - Guía final de acceso, uso y validación del producto

Índice del anexo

1. Finalidad del anexo
2. Alcance operativo del producto entregado
3. Supuestos previos para el uso
4. Superficies de acceso y URLs de la PoC
5. Credenciales y tratamiento seguro de secretos
6. Puesta en marcha y comprobación inicial
7. Obtención y uso del token JWT
8. Uso de la API de Clientes
9. Uso de la API de Cuentas
10. Pruebas de seguridad y control de acceso
11. Uso de RabbitMQ para verificar la mensajería asíncrona
12. Uso de Prometheus y Grafana
13. Consulta de logs y trazabilidad básica
14. Guion recomendado de demostración
15. Resolución de incidencias habituales
16. Checklist final de evidencias de acceso y uso
17. Criterios de aceptación y trazabilidad de evidencias
18. Limitaciones, supuestos y cierre del anexo
19. Referencias bibliográficas

1. Finalidad del anexo

Este anexo documenta el procedimiento de acceso y uso del producto desarrollado en el Trabajo de Fin de Grado. Su finalidad es proporcionar a la persona evaluadora una guía operativa para comprobar el funcionamiento de la prueba de concepto (PoC), ejecutar los casos de uso principales y obtener evidencias de seguridad, observabilidad y mensajería asíncrona.

El documento se sitúa después de los anexos de instalación y configuración. No repite el detalle de despliegue ni la parametrización completa, sino que parte del supuesto de que la PoC está instalada, configurada y levantada correctamente. El foco de este anexo es el uso práctico del producto: qué URLs consultar, cómo obtener un token, cómo invocar las APIs, cómo provocar respuestas 200/401/403/429, cómo verificar RabbitMQ y cómo visualizar métricas en Prometheus y Grafana.

Criterio de seguridad documental: este anexo no contiene secretos reales, tokens completos ni credenciales productivas. Todos los valores sensibles se representan mediante marcadores o deben consultarse exclusivamente en el fichero `.env local` del laboratorio.

Desde una perspectiva académica, este anexo complementa la memoria al transformar la implementación técnica en una guía de verificación reproducible. Para ello, cada operación propuesta queda asociada a un resultado esperado y a una evidencia observable, de forma que el producto pueda evaluarse no solo por su existencia, sino por su comportamiento funcional, de seguridad y de observabilidad.

Criterio de evaluación: una prueba se considera válida cuando la llamada se ejecuta a través de Kong Gateway, devuelve el código HTTP previsto, no expone secretos y deja una evidencia verificable en logs, métricas, interfaz de administración o respuesta JSON.

2. Alcance operativo del producto entregado

El producto entregado es una PoC académica desplegable en entorno local mediante Docker Compose. La solución permite consumir dos APIs REST protegidas por Kong Gateway y Microsoft Entra ID, comprobar autorización diferencial por roles, observar métricas operativas y validar un flujo asíncrono de publicación y consumo de eventos asociado al alta de clientes.

Bloque	Componente	Uso en este anexo
API Gateway	Kong Gateway CE	Punto único de entrada para consumidores. Valida JWT, aplica <i>rate limiting</i> y expone métricas.
Identidad	Microsoft Entra ID	Emisión de JWT RS256 mediante flujo <i>client credentials</i> .
Dominio clientes	CientesApi .NET 8	Alta y consulta de clientes, publicación de evento ClienteCreado.
Dominio cuentas	CuentasApi .NET 8	Consulta de cuentas y saldos.
Mensajería	RabbitMQ + Worker .NET	Broker AMQP, cola cliente-creado y consumidor asíncrono.
Observabilidad	Prometheus + Grafana	Métricas del <i>gateway</i> , códigos HTTP, latencia, tráfico y estado.

Tabla V.1. Alcance operativo del producto

3. Supuestos previos para el uso

Antes de seguir esta guía deben cumplirse las siguientes condiciones:

- La máquina virtual o equipo local dispone de Docker Engine y Docker Compose operativos.
- El repositorio del proyecto está clonado y situado en la rama o versión entregada para evaluación.

- El fichero `.env` existe localmente y contiene los valores de Entra ID, Kong, RabbitMQ y Grafana necesarios para la PoC.
- Los servicios se han levantado con `docker compose up -d` y se encuentran en estado Up o healthy.
- Las aplicaciones registradas en Microsoft Entra ID tienen asignados los app roles correspondientes.
- El gateway se ha configurado con servicios, rutas, plugins, consumers y credenciales JWT.

Relación con anexos anteriores: si alguna de estas condiciones no se cumple, debe revisarse primero el Anexo T de instalación y el Anexo U de configuración técnica.

4. Superficies de acceso y URLs de la PoC

La Tabla V.2 resume los puntos de acceso principales. Las URLs corresponden al entorno local estándar utilizado en la PoC. Si se ejecuta en una máquina virtual, deben sustituirse localhost por la IP o nombre DNS de la máquina, siempre que los puertos estén publicados y accesibles desde el equipo desde el que se realicen las pruebas.

Superficie	URL	Uso
Kong Proxy	<code>http://localhost:8000</code>	Consumo normal de APIs. Es el único punto de entrada funcional.
Kong Admin API	<code>http://localhost:8001</code>	Administración local de servicios, rutas, <i>plugins</i> y <i>consumers</i> . Solo laboratorio.
Kong status/metrics	<code>http://localhost:8100/metrics</code>	Métricas Prometheus expuestas por Kong.
Cientes vía Kong	<code>http://localhost:8000/clientes</code>	Listado y alta de clientes según rol.
Cuentas vía Kong	<code>http://localhost:8000/cuentas</code>	Listado de cuentas según rol.
Prometheus	<code>http://localhost:9090</code>	Consulta de métricas y estado de targets.
Grafana	<code>http://localhost:3000</code>	<i>Dashboard</i> operativo de la PoC.
RabbitMQ Management	<code>http://localhost:15672</code>	Verificación de cola cliente-creado y eventos.
APIs directas	<code>http://localhost:8081 /</code> <code>http://localhost:8082</code>	Solo diagnóstico. Las pruebas de validación deben pasar por Kong.

Tabla V.2. Superficies de acceso del producto

Advertencia: las pruebas funcionales, de seguridad y de observabilidad deben ejecutarse contra el puerto 8000 de Kong. El acceso directo a los backends se reserva para diagnóstico técnico, ya que evita la capa de gobierno del gateway.

5. Credenciales y tratamiento seguro de secretos

La PoC utiliza credenciales locales de laboratorio y secretos de Entra ID. Estos valores no deben incluirse en capturas, memoria, anexos ni repositorio. Cuando sea necesario demostrar su existencia, se debe mostrar el nombre de la variable o el campo de configuración con el valor oculto.

Ámbito	Variable	Ubicación	Tratamiento
Microsoft Entra ID	ENTRA_CLIENT_SECRET	Fichero <code>.env</code> local	No mostrar. Regenerar si se sospecha exposición.
RabbitMQ	RABBITMQ_DEFAULT_USER / RABBITMQ_DEFAULT_PASS	Fichero <code>.env</code> local	Credenciales de laboratorio. No reutilizar.
Grafana	GF_SECURITY_ADMIN_USER / GF_SECURITY_ADMIN_PASSWORD	Fichero <code>.env</code> o configuración Docker	Cambiar tras primer acceso si se usa default.
PostgreSQL	KONG_PG_PASSWORD	Fichero <code>.env</code> local	Solo para la base de configuración de Kong.

Ámbito	Variable	Ubicación	Tratamiento
JWT	TOKEN	Variable temporal de shell	No pegar completo en documentos. Mostrar solo prefijo truncado.

Tabla V.3. Tratamiento de credenciales y secretos

6. Puesta en marcha y comprobación inicial

Desde la raíz del repositorio del proyecto, ejecutar los siguientes comandos para levantar y comprobar el entorno:

```
# 1. Acceder a la raíz del repositorio
cd ~/tfg-plataforma-segura-apis

# 2. Cargar variables locales, si el script de token las necesita
set -a
source .env
set +a

# 3. Levantar la PoC
docker compose up -d

# 4. Comprobar estado de los servicios
docker compose ps

# 5. Revisar logs iniciales si algún servicio no queda operativo
docker compose logs --tail=80 kong
docker compose logs --tail=80 clientes-api
docker compose logs --tail=80 cuentas-api
```

Verificación	Resultado esperado	Propósito
docker compose ps	Servicios en estado Up/healthy	Confirma arranque del entorno.
curl http://localhost:8100/status	Respuesta de Kong Status API	Confirma disponibilidad operativa del <i>gateway</i> .
curl http://localhost:8100/metrics	Texto de métricas Prometheus	Confirma plugin Prometheus y puerto 8100.
curl http://localhost:9090/-/ready	Prometheus is Ready	Confirma Prometheus operativo.
Acceso a http://localhost:3000	Pantalla de login Grafana	Confirma visualización disponible.
Acceso a http://localhost:15672	Pantalla RabbitMQ Management	Confirma <i>broker</i> operativo.

Tabla V.4. Comprobaciones iniciales

7. Obtención y uso del token JWT

Las APIs protegidas requieren un token JWT emitido por Microsoft Entra ID. La forma recomendada de obtenerlo es usar el script `get_token.sh`, que encapsula la llamada OAuth 2.0 client credentials utilizando las variables definidas en `.env`.

```
# Obtener token con el cliente principal de pruebas
```

```
export TOKEN=$(./get_token.sh | tail -1)

# Verificar que se ha recibido un valor sin mostrarlo completo
printf "JWT recibido: %.40s...\n" "$TOKEN"

# Uso estándar en llamadas posteriores
-H "Authorization: Bearer $TOKEN"
```

7.1. Obtención de token para cliente limitado

Para demostrar la autorización diferencial, se recomienda disponer de un segundo conjunto de variables o un segundo script que solicite token para el cliente limitado tfg-api-client-clientes. Este cliente debe tener únicamente el rol Clientes.Read.App.

```
# Ejemplo orientativo si existe un script específico
export TOKEN_LIMITADO=$(./get_token_clientes.sh | tail -1)
printf "JWT limitado recibido: %.40s...\n" "$TOKEN_LIMITADO"
```

Validación esperada: el token completo debe permitir consultar clientes y cuentas, el token limitado debe permitir clientes, pero debe recibir 403 Forbidden al intentar acceder a cuentas.

7.2. Validación mínima del contenido del JWT

Antes de utilizar un token en las pruebas, puede comprobarse de forma local que contiene las *claims* esperadas sin copiar el JWT completo en la documentación. Esta comprobación no sustituye a la validación criptográfica de Kong, pero ayuda a diagnosticar errores de roles, audiencia, aplicación emisora o caducidad.

```
# Comprobación local de claims sin mostrar el token completo
python3 - <<'PY'
import os, json, base64, datetime
token = os.environ.get("TOKEN", "")
parts = token.split(".")
payload = parts[1] + "=" * (-len(parts[1]) % 4)
claims = json.loads(base64.urlsafe_b64decode(payload))
print("appid/azp:", claims.get("appid") or claims.get("azp"))
print("aud:", claims.get("aud"))
print("roles:", claims.get("roles"))
print("exp:", datetime.datetime.fromtimestamp(claims.get("exp", 0)))
PY
```

Resultado esperado: el token debe mostrar la aplicación cliente correcta, la audiencia de la API protegida, los roles necesarios para el caso de prueba y una fecha de expiración vigente. El valor completo del JWT no debe incorporarse a la memoria ni a los anexos.

8. Uso de la API de Clientes

La API de Clientes permite consultar clientes y dar de alta nuevos registros simulados. Todas las llamadas se realizan a través de Kong Gateway.

Método	Ruta pública vía Kong	Rol requerido	Finalidad
GET	/clientes	Clientes.Read.App	Listado de clientes.
GET	/clientes/{id}	Clientes.Read.App	Consulta de cliente por identificador.
POST	/clientes	Clientes.Write.App	Alta de cliente y publicación de evento ClienteCreado.

Tabla V.5. Operaciones de la API de Clientes

8.1. Listado de clientes

```
curl -i \
  -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/clientes
```

Resultado esperado: HTTP 200 OK y un array JSON con clientes simulados.

8.2. Consulta de cliente por identificador

```
curl -i \
  -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/clientes/CLI-A29C6C34
```

Resultado esperado: HTTP 200 OK si el cliente existe, HTTP 404 si el identificador no existe.

8.3. Alta de cliente

La operación de alta requiere el rol Clientes.Write.App. Además de crear el cliente en memoria, la implementación ampliada publica un evento ClienteCreado en RabbitMQ.

```
curl -i -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  http://localhost:8000/clientes \
  -d '{
    "nombre": "Antonio",
    "apellido": "Martos",
    "segmento": "premium"
  }'
```

Resultado esperado: HTTP 201 Created, objeto JSON del cliente creado y, en la versión con RabbitMQ, indicador de evento publicado o evidencia posterior en la cola cliente-creado.

9. Uso de la API de Cuentas

La API de Cuentas permite consultar cuentas y saldos simulados. Requiere el rol Cuentas.Read.App.

Método	Ruta pública vía Kong	Rol requerido	Finalidad
GET	/cuentas	Cuentas.Read.App	Listado de cuentas.
GET	/cuentas/{id}	Cuentas.Read.App	Consulta de una cuenta.
GET	/cuentas/{id}/saldo	Cuentas.Read.App	Consulta de saldo.

Tabla V.6. Operaciones de la API de Cuentas

9.1. Listado de cuentas

```
curl -i \
  -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/cuentas
```

Resultado esperado: HTTP 200 OK y array JSON de cuentas simuladas.

9.2. Consulta de saldo

```
curl -i \
  -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/cuentas/ACC-001/saldo
```

Resultado esperado: HTTP 200 OK con el saldo y la moneda de la cuenta. Si el identificador no existe, la API debe devolver HTTP 404.

10. Pruebas de seguridad y control de acceso

Estas pruebas demuestran que la PoC no solo expone servicios, sino que aplica autenticación, autorización y control de tráfico de forma verificable.

Prueba	Ejecución	Resultado esperado	Evidencia
Sin token	Llamada GET a clientes sin Authorization	401 Unauthorized	Autenticación obligatoria.
Token inválido	Authorization: Bearer token_invalido	401 Unauthorized	Validación JWT en Kong.
Token limitado	Token con Clientes.Read.App contra cuentas	403 Forbidden	Autorización diferencial por roles.
Token completo	Token con rol adecuado	200 OK	Acceso permitido.
Rate limiting	>10 peticiones/minuto	429 Too Many Requests	Control de tráfico en Kong.

Tabla V.7. Pruebas de seguridad recomendadas

10.1. Sin token

```
curl -i http://localhost:8000/clientes
```

10.2. Token inválido

```
curl -i \
  -H "Authorization: Bearer token_invalido" \
  http://localhost:8000/clientes
```

10.3. Token limitado contra cuentas

```
curl -i \
  -H "Authorization: Bearer $TOKEN_LIMITADO" \
  http://localhost:8000/cuentas
```

10.4. Rate limiting

```
for i in $(seq 1 15); do
  echo "Petición $i"
  curl -s -o /dev/null -w "%{http_code}\n" \
    -H "Authorization: Bearer $TOKEN" \
    http://localhost:8000/clientes
done
```

Resultado esperado: las primeras peticiones deben devolver 200 y, al superar el umbral configurado, Kong debe devolver 429.

10.5. Relación con OWASP API Security Top 10

Las pruebas anteriores no sustituyen a una auditoría de seguridad completa, pero permiten demostrar una cobertura razonable de controles relevantes para una PoC académica de APIs. En particular, se relacionan con los siguientes riesgos de OWASP API Security Top 10:

- API1:2023 - Broken Object Level Authorization: se cubre de forma parcial, ya que la PoC demuestra autorización por dominio y operación, pero no autorización a nivel de objeto individual.

- API2:2023 - Broken Authentication: se valida con peticiones sin token, token inválido y token caducado o no aceptado por Kong.
- API4:2023 - Unrestricted Resource Consumption: se cubre mediante la política de rate limiting y la obtención controlada de 429 Too Many Requests.
- API8:2023 - Security Misconfiguration: se revisa mediante la existencia de rutas, plugins, métricas y configuración de acceso diferenciada entre Kong, APIs y componentes de observabilidad.
- API10:2023 - Unsafe Consumption of APIs: no aplica directamente al alcance operativo, porque la PoC no consume APIs externas de terceros ni tráfico productivo, se mantiene como consideración de diseño futuro.

Criterio de cierre: las evidencias de 401, 403, 429 y métricas asociadas deben incorporarse a la memoria o al volumen de anexos para demostrar que la seguridad no se describe solo de forma conceptual, sino que se ha probado en ejecución.

11. Uso de RabbitMQ para verificar la mensajería asíncrona

El flujo asíncrono se valida generando un alta de cliente y verificando que el evento ClienteCreado se publica en la cola cliente-creado y es consumido por el Worker Service .NET.

1. Acceder a RabbitMQ Management en <http://localhost:15672>.
2. Autenticarse con el usuario y contraseña definidos en .env.
3. Localizar la cola cliente-creado.
4. Ejecutar un POST de alta de cliente con token que incluya Clientes.Write.App.
5. Comprobar que la cola recibe el mensaje o que el consumidor lo procesa.
6. Consultar los logs del consumidor para verificar el evento recibido.

```
# Logs del consumidor asíncrono
docker compose logs -f cliente-creado-consumer

# Logs de ClientesApi para comprobar publicación del evento
docker compose logs --tail=100 clientes-api
```

Evidencia	Dónde verificar	Resultado esperado
Alta correcta	POST /clientes	201 Created
Cola visible	RabbitMQ Management	cliente-creado existe
Evento publicado	Mensajes Ready/Total o log de publicación	Evento ClienteCreado generado
Evento consumido	Logs del Worker	Evento ClienteCreado recibido + BasicAck

Tabla V.8. Evidencias del flujo asíncrono

12. Uso de Prometheus y Grafana

12.1. Prometheus

Prometheus permite consultar directamente las métricas expuestas por Kong. Tras generar tráfico, las consultas deben mostrar valores crecientes.

```
# Comprobar que Kong es target activo
# En navegador: http://localhost:9090/targets

# Consultas útiles en Prometheus
```

```

rate(kong_nginx_requests_total[1m])
sum by (code)(rate(kong_http_requests_total[5m]))
sum by (code)(increase(kong_http_requests_total[$__range]))
kong_http_requests_total{code=~"401|403|429"}
kong_http_requests_total{code=~"5.."}
kong_datastore_reachable
up{job="kong"}

```

12.2. Grafana

Grafana es la interfaz recomendada para mostrar la observabilidad durante la defensa. El *dashboard* debe reflejar tráfico real generado durante las pruebas: llamadas correctas, rechazos 401/403, bloqueos 429 y estado del *gateway*.

Panel	Cómo generar datos	Resultado esperado
Kong - Tasa de peticiones	Generar varias llamadas GET/POST vía Kong	Serie temporal con incremento de peticiones por segundo.
Kong - Respuestas HTTP por código	Ejecutar llamadas 200, 401, 403 y 429	Distribución temporal por código HTTP usando <code>kong_http_requests_total</code> .
Kong - Total de respuestas por código HTTP	Generar tráfico en el rango del dashboard	Conteo agregado por código HTTP en el periodo seleccionado.
Kong - Errores técnicos 5xx	Mantener backends operativos durante la prueba	Valor esperado 0 si no hay fallo técnico de plataforma.
Kong - Peticiones rechazadas o limitadas	Probar sin token, token limitado y rate limiting	Porcentaje de 401/403/429 sobre el total de peticiones.
Kong - Estado del datastore	Verificar gateway con PostgreSQL disponible	Valor 1 si el datastore de Kong está accesible.

Tabla V.9. Uso del dashboard de Grafana

13. Consulta de logs y trazabilidad básica

Los logs son útiles para diagnosticar incidencias y demostrar trazabilidad básica entre *gateway* y *backend*. La PoC utiliza logs de Kong, logs de APIs .NET y logs del consumidor RabbitMQ.

```

# Logs de Kong: método, ruta, status y latencia
docker compose logs -f kong

# Logs de APIs backend
docker compose logs -f clientes-api
docker compose logs -f cuentas-api

# Logs del consumidor de eventos
docker compose logs -f cliente-creado-consumer

# Filtrar posibles rechazos de seguridad
docker compose logs kong | grep -E " 401 | 403 | 429 "

```

Origen	Información relevante	Uso en la defensa
Kong	Entrada de la petición, código HTTP, consumer, latencia	Confirmar que el tráfico pasa por <i>gateway</i> .

Origen	Información relevante	Uso en la defensa
CientesApi	Autorización por rol, alta de cliente, publicación de evento	Confirmar comportamiento funcional.
CuentasApi	Autorización por rol y consultas de cuentas/saldo	Confirmar acceso permitido o denegado.
ClienteCreadoConsumer	Recepción y confirmación del evento	Confirmar flujo asíncrono.

Tabla V.10. Fuentes de logs y uso

14. Guion recomendado de demostración

Se recomienda ejecutar una secuencia breve que muestre valor funcional, seguridad y observabilidad sin entrar en tareas de instalación.

Paso	Demostración	Acción	Mensaje clave
1	Estado de contenedores	docker compose ps	Entorno operativo.
2	Token válido	./get_token.sh	Identidad federada activa.
3	Consulta clientes	GET /clientes	200 OK.
4	Consulta cuentas	GET /cuentas	200 OK con cliente completo.
5	Acceso sin token	GET sin Authorization	401 Unauthorized.
6	Token limitado	GET cuentas con TOKEN_LIMITADO	403 Forbidden.
7	Alta cliente	POST /clientes	201 + evento ClienteCreado.
8	RabbitMQ	Cola y logs del consumer	Evento procesado.
9	Rate limiting	Bucle de 15 peticiones	429.
10	Grafana	Dashboard	Métricas con datos reales.

Tabla V.11. Guion recomendado de demostración

15. Resolución de incidencias habituales

Incidencia	Causa probable	Resolución
No carga Kong Proxy	Contenedor kong parado o puerto 8000 ocupado	docker compose ps, revisar logs, liberar puerto.
401 con token aparentemente correcto	Consumer JWT no coincide con appid o clave pública incorrecta	Revisar credencial JWT en Kong y kid de Entra ID.
403 en operaciones esperadas	Falta app role en el token	Revisar asignación de roles y obtener nuevo token.
404 en ruta	Ruta pública no coincide con configuración strip_path	Consultar rutas en Admin API y ajustar URL pública.
No aparece 429	Umbral no superado o plugin no activo	Verificar plugin rate-limiting y repetir bucle de llamadas.
RabbitMQ no muestra mensajes	Consumer los consume inmediatamente o POST no publicó evento	Revisar logs de ClientesApi y del Worker.
Prometheus sin datos	Target kong:8100 down	Revisar prometheus.yml y endpoint /metrics.
Grafana sin paneles	Datasource mal configurado o dashboard no importado	Probar datasource Prometheus y cargar dashboard JSON.

Tabla V.12. Incidencias habituales de acceso y uso

16. Checklist final de evidencias de acceso y uso

Código	Evidencia	Contenido mínimo
E-USO-01	docker compose ps	Todos los servicios principales en estado Up/healthy.
E-USO-02	Kong Proxy	Acceso al puerto 8000 con respuesta controlada.
E-USO-03	Token JWT	Token obtenido desde Entra ID, mostrado truncado.
E-USO-04	GET clientes	HTTP 200 con array JSON.
E-USO-05	POST clientes	HTTP 201 y cliente creado.
E-USO-06	GET cuentas	HTTP 200 con array JSON.
E-USO-07	GET saldo	HTTP 200 con saldo y moneda.
E-USO-08	401	Petición sin token o token inválido rechazada.
E-USO-09	403	Token limitado rechazado en cuentas.
E-USO-10	429	<i>Rate limiting</i> activado tras superar umbral.
E-USO-11	RabbitMQ	Cola cliente-creado y evento procesado.
E-USO-12	Prometheus	Target Kong UP y métricas visibles.
E-USO-13	Grafana	<i>Dashboard</i> con tráfico y códigos HTTP.
E-USO-14	Logs	Evidencias en Kong, APIs y <i>consumer</i> .

Tabla V.13. Checklist final de evidencias de acceso y uso

17. Criterios de aceptación y trazabilidad de evidencias

Esta sección consolida los criterios que permiten considerar válido el acceso y uso del producto. Su objetivo es evitar que la demostración dependa únicamente de capturas aisladas y establecer una correspondencia clara entre operación ejecutada, resultado esperado y evidencia verificable.

17.1. Criterios de aceptación del uso del producto

Código	Criterio de aceptación	Evidencia mínima
CA-USO-01	La PoC arranca de forma reproducible desde la raíz del repositorio.	docker compose ps con servicios principales en estado Up/healthy.
CA-USO-02	Kong Gateway actúa como punto único de entrada funcional.	Llamadas correctas a http://localhost:8000 y uso directo de <i>backends</i> reservado a diagnóstico.
CA-USO-03	La autenticación se basa en JWT emitido por Microsoft Entra ID.	Token obtenido por client credentials y mostrado únicamente truncado.
CA-USO-04	La autorización diferencia consumidores con distintos roles.	200 con token completo y 403 con token limitado en operaciones no autorizadas.
CA-USO-05	El control de tráfico está activo en el <i>gateway</i> .	Obtención de 429 al superar el umbral configurado.
CA-USO-06	El flujo asíncrono asociado al alta de cliente es observable.	POST /clientes, cola cliente-creado y logs del consumidor.
CA-USO-07	Las métricas del <i>gateway</i> son visibles en Prometheus y Grafana.	Target Kong UP, métricas PromQL y <i>dashboard</i> con tráfico real.
CA-USO-08	La documentación no expone secretos ni credenciales reutilizables.	Capturas y comandos con valores sensibles ocultos o truncados.

Tabla V.14. Criterios de aceptación del acceso y uso del producto

17.2. Trazabilidad entre evidencias, requisitos y validación

Evidencia	Requisito o dimensión validada	Interpretación para la memoria
E-USO-01 a E-USO-02	Reproducibilidad y exposición gobernada	Demuestra que la PoC puede arrancarse y utilizarse desde un entorno controlado.

Evidencia	Requisito o dimensión validada	Interpretación para la memoria
E-USO-03	Autenticación federada	Acredita la integración con Entra ID y el uso de OAuth 2.0/JWT.
E-USO-04 a E-USO-07	Funcionalidad de ClientesApi y CuentasApi	Evidencia que los casos de uso principales están implementados.
E-USO-08 a E-USO-10	Seguridad de APIs y control de acceso	Demuestra autenticación obligatoria, autorización diferencial y limitación de tráfico.
E-USO-11	Integración asíncrona parcial	Acredita el valor demostrativo de RabbitMQ sin convertirlo en alcance principal.
E-USO-12 a E-USO-13	Observabilidad	Permite justificar que el <i>gateway</i> no es una caja negra y que existen métricas operativas.
E-USO-14	Trazabilidad básica	Facilita el diagnóstico técnico y la defensa ante incidencias reproducibles.

Tabla V.15. Trazabilidad entre evidencias, requisitos y validación

17.3. Reglas para la captura de evidencias

Para que las evidencias sean útiles en la evaluación final, se recomienda aplicar las siguientes reglas de captura y conservación:

- Mostrar siempre el comando, la URL o el panel utilizado, junto con el resultado observable.
- Ocultar secretos, tokens completos, client secrets, contraseñas y valores del fichero .env.
- Conservar el código HTTP, la fecha aproximada de ejecución y el componente que genera la evidencia.
- Evitar capturas redundantes: cada imagen debe justificar una validación concreta del producto.
- Referenciar desde la memoria únicamente las evidencias más representativas, dejando el detalle operativo en el anexo.

La aplicación de estas reglas refuerza la calidad formal del TFG porque conecta la demostración técnica con criterios de evaluación verificables, reduce ruido documental y evita riesgos de exposición de información sensible.

18. Limitaciones, supuestos y cierre del anexo

El uso del producto debe interpretarse dentro del alcance definido para la PoC académica. La solución valida una arquitectura de transición y sus controles principales, pero no constituye un despliegue productivo ni una implantación completa en infraestructura bancaria real.

- El entorno de ejecución es Docker local o una máquina virtual de laboratorio, no AKS ni un entorno *cloud* productivo.
- Los datos de clientes, cuentas y saldos son simulados y no representan información bancaria real.
- RabbitMQ se utiliza como integración avanzada demostrativa, no se implementa una arquitectura completa de eventos de negocio.
- El acceso directo a los *backends* solo se admite para diagnóstico técnico y no forma parte del patrón de consumo recomendado.
- No se implementa SSO final entre plataformas ni *frontend* de usuario, ambos elementos pertenecen a la arquitectura objetivo o a líneas futuras.
- La validación OWASP se plantea como *checklist* razonado de controles aplicables a la PoC, no como *pentest* externo certificado.

Estas limitaciones no reducen el valor del producto entregado, al contrario, delimitan con precisión el alcance y evitan atribuir a la PoC capacidades propias de una solución productiva que no han sido objeto de implementación.

La ejecución de las pruebas descritas en este anexo permite comprobar el uso completo del producto entregado: acceso a las APIs a través de Kong, obtención de tokens desde Entra ID, autorización por roles, control de tráfico, publicación y consumo de eventos en RabbitMQ y observabilidad mediante Prometheus y Grafana.

El anexo cumple así una doble función: por un lado, facilita la revisión técnica de la PoC por parte del tribunal, por otro, proporciona una guía de demostración reproducible que conecta directamente el producto con los resultados presentados en la memoria.

Recomendación final: antes de la entrega definitiva, incorporar capturas reales de las evidencias E-USO-01 a E-USO-14 o, alternativamente, referenciar las capturas ya incluidas en el volumen de anexos técnicos.

19. Referencias bibliográficas

Docker Inc. (s. f.-a). Docker Compose. Docker Docs. <https://docs.docker.com/compose/>

Docker Inc. (s. f.-b). docker compose up. Docker Docs. <https://docs.docker.com/reference/cli/docker/compose/up/>

Grafana Labs. (s. f.). Prometheus data source. Grafana documentation. <https://grafana.com/docs/grafana/latest/datasources/prometheus/>

Hardt, D. (2012). The OAuth 2.0 Authorization Framework (RFC 6749). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6749>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>

Kong Inc. (s. f.-a). JWT authentication plugin. Kong Docs. <https://developer.konghq.com/plugins/jwt/>

Kong Inc. (s. f.-b). Prometheus plugin. Kong Docs. <https://developer.konghq.com/plugins/prometheus/>

Kong Inc. (s. f.-c). Rate Limiting plugin. Kong Docs. <https://developer.konghq.com/plugins/rate-limiting/>

Microsoft. (2026, 30 de enero). OAuth 2.0 client credentials flow on the Microsoft identity platform. Microsoft Learn. <https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-client-creds-grant-flow>

OWASP Foundation. (2023). OWASP API Security Top 10 - 2023. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

Prometheus Authors. (s. f.). Querying basics. Prometheus documentation. <https://prometheus.io/docs/prometheus/latest/querying/basics/>

RabbitMQ. (s. f.). Management plugin. RabbitMQ documentation. <https://www.rabbitmq.com/docs/management>